

姓名： 张洋

学号： 20232411

《卫星导航定位基础》课后作业（二）

GNSS 载波相位平滑伪距单点定位

任务部分：ftp://igs.gnsswhu.cn/pub/gps/data/daily/2025/) 下载一个不同天、不同测站名的 RINEX 观测值 o 文件，利用载波相位平滑伪距，实现批量测站位置的计算并进行精度评定，对比分析载波相位平滑伪距前后的定位精度变化。

一、 算法

在全球卫星导航系统（GNSS）定位技术的演进历程中，载波相位平滑伪距单点定位算法凭借其卓越的性能，成为实现高精度定位的关键技术之一。而 Hatch 滤波作为该算法的核心组成部分，通过巧妙的数据处理策略，显著提升了定位的精准度与稳定性。

伪距测量是 GNSS 定位的基础操作，其原理基于信号传播的时间延迟。卫星以光速 c 发射信号，接收机接收到信号时存在时间延迟 Δt ，由此可计算出卫星到接收机的伪距 P ，即 $P = c \cdot \Delta t$ 。在实际环境中，多种误差源严重影响伪距测量精度。卫星轨道并非理想状态，存在轨道误差 δ_{orb} ；信号在穿越电离层时，由于电离层的折射效应，会产生与频率相关的延迟 δ_{ion} ，对于双频信号（频率分别

为 f_1 和 f_2)，可通过组合观测值进行一阶电离层延迟改正，改正后的电离层延迟近似为

$$\delta_{ion}^{corr} \approx \frac{f_1^2}{f_1^2 - f_2^2} \delta_{ion1} - \frac{f_2^2}{f_1^2 - f_2^2} \delta_{ion2},$$

其中 δ_{ion1} 和 δ_{ion2} 分别是频率 f_1 和 f_2 对应的电离层延迟。信号穿越对流层时，受大气折射影响产生对流层延迟 δ_{trop} ，一般可采用经验模型（如 Saastamoinen 模型）进行估算，其表达式为

$$\delta_{trop} = \frac{0.002277}{\cos z} \left(P_s / T_s + \frac{e_s}{T_s^2} \right) h$$

式中 z 为卫星天顶距， P_s 、 T_s 、 e_s 分别为测站处的气压、气温和水汽压， h 为测站海拔高度。此外，多路径效应导致信号经不同路径传播后到达接收机，产生多路径误差 δ_{mp} 。综合这些误差，实际测量的伪距 P_{obs} 可表示为

$$P_{obs} = \rho + c \cdot (\delta_{orb} + \delta_{ion}^{corr} + \delta_{trop} + \delta_{mp}) + c \cdot \delta_{clk}^r + c \cdot \delta_{clk}^s + \varepsilon_p$$

其中 ρ 为卫星到接收机的真实几何距离， δ_{clk}^r 和 δ_{clk}^s 分别为接收机钟差和卫星钟差， ε_p 为伪距测量噪声。这些误差的综合影响使得单纯依靠伪距测量的定位精度通常只能达到米级，难以满足高精度应用需求。

载波相位测量利用 GNSS 信号中的载波进行测量，具有极高的精度潜力。载波相位观测值 φ 的数学表达式为 $\varphi = \frac{\rho}{\lambda} + N + \varepsilon$ ，其中 ρ 代表卫星到接收机的真实几何距离， λ 是载波波长（例如 GPS L1 载波波长 λ_{L1} 0.1903 米）， N 为整周模糊度，是一个整数，但在

初始测量时难以直接确定， ε 表示观测噪声，其噪声水平相较于伪距测量极低，理论上载波相位测量具备毫米级的精度潜力。然而，整周模糊度的精确解算一直是载波相位测量应用中的难题，需要复杂的算法和大量的观测数据支持。在实际应用中，为了消除或削弱部分误差，常采用双差观测值。设两个接收机 r_1 和 r_2 同时观测卫星 s_1 和 s_2 ，双差载波相位观测值 $\Delta \Delta \varphi$ 可表示为

$$\Delta \Delta \varphi = (\varphi_{r_1}^{s_1} - \varphi_{r_1}^{s_2}) - (\varphi_{r_2}^{s_1} - \varphi_{r_2}^{s_2})$$

双差观测值在一定程度上消除了卫星钟差、接收机钟差以及部分大气延迟误差，为后续的高精度定位计算提供了更可靠的数据基础。

该算法的核心思想是利用载波相位测量的高精度特性来优化伪距测量结果。由于载波相位测量噪声小，对连续历元的载波相位观测值进行积分，能够获取高精度的距离变化信息。设相邻两个历元 k 和 $k + 1$ 的载波相位观测值分别为 φ_k 和 φ_{k+1} ，则这两个历元间的距离变化量 $\Delta \rho$ 可通过公式

$$\Delta \rho = \lambda(\varphi_{k+1} - \varphi_k)$$

计算得出。在实际应用中，将此距离变化量与伪距测量值相结合，能有效降低伪距测量中的噪声和误差影响，从而提升定位精度。具体实现时，需要将载波相位观测值转换为与伪距具有相同量纲的距离值，设经过整周模糊度解算和其他误差改正后的载波相位

对应的距离为 ρ_k^s ，其计算过程较为复杂，涉及到对初始整周模糊度的确定以及各种误差的精确改正，这里简化表示为

$$\rho_k^s = \lambda(\varphi_k - N_k^{corr}) + \rho_0$$

其中 N_k^{corr} 为经过改正后的整周模糊度， ρ_0 为初始参考距离。通过不断更新 ρ_k^s 并与伪距观测值融合，实现对伪距的平滑处理。

Hatch 滤波，即滑动平均滤波，是一种简单且有效的线性滤波方法。在载波相位平滑伪距算法中，它通过对连续的载波相位和伪距观测值进行加权平均，来逐步提高伪距测量的精度。其基本思想是给予近期观测值更高的权重，因为近期观测值更能反映当前的实际情况，而随着观测值时间的推移，权重逐渐降低。Hatch 滤波的核心在于构建一个有限长度的观测值滑动窗口，对窗口内的观测值进行加权处理，从而得到更准确的估计值。

设 P_k 为第 k 历元的伪距观测值， ρ_k^s 为根据载波相位观测值推算出的第 k 历元卫星到接收机的距离（通过对载波相位积分并考虑整周模糊度得到）， $\widehat{P}_k$ 为经过 Hatch 滤波后的伪距估计值。Hatch 滤波的递推公式如下：

$$\widehat{P}_k = \frac{n-1}{n} \widehat{P}_{k-1} + \frac{1}{n} \rho_k^s$$

其中， n 为滤波窗口大小，它决定了参与平均的观测值数量。窗口大小的选择至关重要， n 过小，滤波效果不明显，难以有效平滑噪声； n 过大，则对信号变化的响应速度变慢，可能导致在接收

机动态变化时无法及时跟踪真实位置变化。为了更深入理解滤波过程，我们可以将其展开。假设初始时刻 $k = 0$ ，初始化滤波值 $\widehat{P}_0 = P_0$ 。当进入第 1 历元时

$$\widehat{P}_1 = \frac{n-1}{n} \widehat{P}_0 + \frac{1}{n} \rho_1^s = \frac{n-1}{n} P_0 + \frac{1}{n} \rho_1^s$$

随着历元推进到第 2 历元

$$\begin{aligned} \widehat{P}_2 &= \frac{n-1}{n} \widehat{P}_1 + \frac{1}{n} \rho_2^s = \frac{n-1}{n} \left(\frac{n-1}{n} P_0 + \frac{1}{n} \rho_1^s \right) + \frac{1}{n} \rho_2^s \\ &= \left(\frac{n-1}{n} \right)^2 P_0 + \frac{n-1}{n^2} \rho_1^s + \frac{1}{n} \rho_2^s \end{aligned}$$

以此类推，在第 k 历元时， $\widehat{P}_k$ 是一个关于 P_0 以及从 ρ_1^s 到 ρ_k^s 的加权和，其中权重随着历元的远离而呈指数衰减。每一次更新都综合考虑了上一历元的滤波结果和当前历元根据载波相位推算出的距离信息，从而逐步降低伪距测量中的噪声和误差，使伪距估计值更接近真实距离。

在实际应用中，还需要考虑滤波窗口的更新机制。当新的观测值进入窗口时，最旧的观测值会被移除。例如，假设滤波窗口大小 $n = 5$ ，在历元 k 时，窗口内的观测值为

$$[\rho_{k-4}^s, \rho_{k-3}^s, \rho_{k-2}^s, \rho_{k-1}^s, \rho_k^s]$$

计算 $\widehat{P}_k$ 后，当历元更新到 $k+1$ 时，窗口内的观测值变为

$$[\rho_{k-3}^s, \rho_{k-2}^s, \rho_{k-1}^s, \rho_k^s, \rho_{k+1}^s]$$

通过这种动态更新窗口的方式，Hatch 滤波能够实时适应观测数据的变化，保持对伪距的有效平滑。

相较于其他复杂的滤波算法，Hatch 滤波具有计算简单、易于实现的显著优势。它不需要复杂的矩阵运算和大量的计算资源，这使得在一些计算能力有限的设备，如小型无人机、低成本定位模块等中能够高效运行。在稳定的观测环境下，Hatch 滤波能够快速收敛到较为准确的伪距估计值，有效提升定位精度。以静态定位场景为例，经过短暂的观测数据积累和滤波处理，基于 Hatch 滤波的载波相位平滑伪距算法能够将定位精度从单纯伪距定位的米级提升至分米级甚至厘米级。例如，在某静态定位实验中，使用 GPS 接收机进行观测，初始伪距定位精度约为 5 米，经过 10 分钟基于 Hatch 滤波（窗口大小 $n = 10$ ）的载波相位平滑处理后，定位精度提升至 0.2 米以内，满足诸如地质监测、建筑物变形测量等对精度要求较高的应用需求。

基于 Hatch 滤波的载波相位平滑伪距单点定位算法，结合了载波相位测量的高精度和 Hatch 滤波的高效数据处理能力。与单纯的伪距单点定位相比，其定位精度得到了大幅提升，能够满足精密农业中农机精准作业（如精准播种要求定位精度在 0.1 米以内）、无人机精细测绘（需要厘米级定位精度来生成高精度地图）等对高精度定位的严格要求。同时，由于 Hatch 滤波的简单性，该算法在实时性方面表现出色，能够快速响应接收机位置的变化，为实时导航（如车辆导航要求定位结果实时更新且误差较小）、车辆动态监控

等应用提供及时、准确的定位信息。在一些对成本敏感且需要一定定位精度的物联网设备定位场景中，该算法也具有明显优势，能够在低成本硬件平台上实现相对高精度的定位功能。

然而，该算法也存在一定的局限性。在信号遮挡严重的环境，如城市高楼林立的区域或茂密森林中，卫星信号容易受到阻挡，导致载波相位观测值出现失锁、周跳等现象。一旦出现周跳，载波相位积分得到的距离变化信息就会出现错误，进而影响伪距的平滑效果，使定位精度大幅下降。例如，在城市峡谷环境中，当卫星信号被建筑物遮挡导致载波相位失锁 10 秒后重新锁定，重新锁定后的 5 分钟内，基于 Hatch 滤波的定位误差可能会增大到 1 米以上。此外，在多路径效应强烈的区域，如靠近大面积水域或金属反射物附近，信号的多次反射会干扰载波相位观测值，使得根据载波相位推算的距离存在较大误差，即使经过 Hatch 滤波处理，也难以获得高精度的定位结果。在某些靠近湖面的测试区域，多路径效应导致载波相位观测值的误差可达 0.5 米以上，严重影响了最终的定位精度，使定位误差超出分米级精度要求。

二、 代码

星历时间：2025 年 2 月 24 日；

观测站地点：巴黎；

运行环境：python 3.13.2；

库依赖：pandas, numpy, matplotlib, plotly, datetime

源代码结构说明：

1、数据文件：

brdc0550.25n - 2025 年 2 月 24 日广播星历数据源文件；

opmt0550.25o - 2025 年 2 月 24 日巴黎观测数据源文件

2、计算程序：

main.py - 主程序(运行的文件)

HatchFilter.py - 滤波算法程序

ReadFile.py - 文件读取程序

Satelite.py - 卫星位置结算程序

Position.py - 接收站位置计算程序

Result.py - 结果输出为文件程序

Plot.py - 绘图程序

3、输出文件：

xyz.csv - 各个时刻接收站位置计算结果

dop.csv - 各个时刻精度因子计算结果

3d_position.jpg - 各个时刻各个立面的计算位置展示图

residuals.jpg - 各个时刻 xyz 三轴计算值误差分布图

dop_metrics.jpg - 各个时刻精度因子计算结果波动图

main.py

```
from ReadFile import ReadFile
from Position import Position
from Plot import visualize_dop_metrics,
visualize_3d_position, visualize_residuals
from Result import save_results_to_excel

def main():
    file_pairs = [["./input/opmt0550.25o",
                    "./input/brdc0550.25n"]]# 可以加入多个文件

    for file_pair in file_pairs:
        readfile = ReadFile(file_pair)
        readfile.CaculateSatelites()
        position =
Position(readfile.SateliteObservation,
readfile.PosName, readfile.Time,
          readfile.SateliteClockCorrect)
        position.MatchObservationAndCaculate()
        file_name = file_pair[0].split('/')[-1].split('.')[0]
        save_results_to_excel(position, file_name)
        if position.final_positions:
            visualize_3d_position(position)
            visualize_dop_metrics(position)
            visualize_residuals(position)
        else:
            print("No valid positioning results to
visualize.")

if __name__ == "__main__":
    main()
```

HatchFilter.py

```
class HatchFilter:
    def __init__(self, alpha):
        self.alpha = alpha
```

```

        self.previous_value = None
        self.smoothed_values = []

    def filter(self, carrier_phase, pseudorange):
        if self.previous_value is None:
            self.previous_value = pseudorange -
carrier_phase
        else:
            self.previous_value = self.alpha *
self.previous_value + \
                                (1 - self.alpha) *
(pseudorange - carrier_phase)
            self.smoothed_values.append(self.previous_value)
        return self.previous_value

```

Plot.py

```

import matplotlib.pyplot as plt
import numpy as np
import matplotlib.dates as mdates
from datetime import datetime

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

def visualize_dop_metrics(position):
    # 将观测时间字符串转换为 datetime 对象
    observation_times = [datetime.strptime(time,
'%Y-%m-%d %H:%M:%S') for time in
position.observation_times]

    fig, axes = plt.subplots(5, 1, figsize=(8, 15))
    fig.suptitle("DOP 指标")

    dop_metrics = [
        (position.gdop_values, "GDOP", 'green'),
        (position.pdop_values, "PDOP", 'orange'),
        (position.hdop_values, "HDOP", 'blue'),

```

```

        (position.vdop_values, "VDOP", 'red'),
        (position.tdop_values, "TDOP", 'purple')
    ]

    for ax, (values, label, color) in zip(axes,
dop_metrics):
        if values:
            # 设置 x 轴为日期格式
            ax.xaxis_date()
            # 设置 x 轴刻度为一小时一个点
            ax.xaxis.set_major_locator(mdates.HourLocato
r())

            # 设置 x 轴刻度标签格式, 只显示时间且不显示 :
            ax.xaxis.set_major_formatter(mdates.DateForm
atter('%H'))
            # 不旋转 x 轴刻度标签
            plt.setp(ax.get_xticklabels(), rotation=0)
            # 减小坐标点的大小
            ax.plot(observation_times, values,
marker='o', markersize=2, label=label, color=color)
            ax.set_ylabel(label)
            ax.legend()

        plt.tight_layout()
        plt.subplots_adjust(top=0.95)
        # 保存图片为 jpg 格式
        fig.savefig('./output/dop_metrics.jpg',
format='jpg', dpi=300)
        plt.show()

def visualize_residuals(position):
    if position.final_positions and position.ApproxPos:
        final_positions =
np.array(position.final_positions)
        approx_pos = np.array(position.ApproxPos)

        residuals_x = final_positions[:, 0] -
approx_pos[0]

```

```

        residuals_y = final_positions[:, 1] -
approx_pos[1]
        residuals_z = final_positions[:, 2] -
approx_pos[2]

        residuals = [residuals_x, residuals_y,
residuals_z]
        labels = ["X 坐标残差", "Y 坐标残差", "Z 坐标残差"]
        colors = ['blue', 'green', 'red']

        fig, axes = plt.subplots(3, 1, figsize=(8, 12))
        fig.suptitle("定位坐标的 X、Y、Z 残差图")

        for ax, res, label, color in zip(axes,
residuals, labels, colors):
            ax.scatter(np.arange(len(res)), res,
color=color, label=label, alpha=0.5)

            # 绘制 y = 0 参考线
            ax.axhline(y=0, color='black', linestyle='--
', label='残差均值参考线')

            ax.set_ylabel("残差 (m)")
            ax.legend()

        plt.tight_layout()
        # 保存图片为 jpg 格式
        fig.savefig('./output/residuals.jpg',
format='jpg', dpi=300)
        plt.show()

def visualize_3d_position(position):
    if position.final_positions:
        final_positions =
np.array(position.final_positions)
        approx_pos = np.array(position.ApproxPos)

```

```
xs, ys, zs = final_positions[:, 0],
final_positions[:, 1], final_positions[:, 2]

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle("三维位置估计的三个立面平面图")

# XY 平面
axes[0].scatter(xs, ys, c='blue', marker='o',
s=10, label="估计位置")
axes[0].scatter(approx_pos[0], approx_pos[1],
c='red', marker='x', s=100, label="近似位置")
axes[0].plot(xs, ys, linestyle='--',
color='gray', alpha=0.5, label="轨迹")
axes[0].set_xlabel("X (m)")
axes[0].set_ylabel("Y (m)")
axes[0].set_title("XY 平面")
axes[0].legend()

# XZ 平面
axes[1].scatter(xs, zs, c='blue', marker='o',
s=10, label="估计位置")
axes[1].scatter(approx_pos[0], approx_pos[2],
c='red', marker='x', s=100, label="近似位置")
axes[1].plot(xs, zs, linestyle='--',
color='gray', alpha=0.5, label="轨迹")
axes[1].set_xlabel("X (m)")
axes[1].set_ylabel("Z (m)")
axes[1].set_title("XZ 平面")
axes[1].legend()

# YZ 平面
axes[2].scatter(ys, zs, c='blue', marker='o',
s=10, label="估计位置")
axes[2].scatter(approx_pos[1], approx_pos[2],
c='red', marker='x', s=100, label="近似位置")
axes[2].plot(ys, zs, linestyle='--',
color='gray', alpha=0.5, label="轨迹")
axes[2].set_xlabel("Y (m)")
```

```

        axes[2].set_ylabel("Z (m)")
        axes[2].set_title("YZ 平面")
        axes[2].legend()

        plt.tight_layout()
        # 保存图片为 jpg 格式
        fig.savefig('./output/3d_position.jpg',
format='jpg', dpi=300)
        plt.show()
    else:
        print("No valid positioning results to visualize
in 2D planes.")

```

Position.py

```

import datetime
import numpy as np
from HatchFilter import HatchFilter
from ReadFile import ReadFile
from Satelite import Satelite

class Position:
    def __init__(self, SateliteObservation,
SateliteName, Time, SateliteClockCorrect):
        self.SateliteObservation = SateliteObservation
        self.SateliteName = SateliteName
        self.Time = Time
        self.SateliteClockCorrect = SateliteClockCorrect
        self.Lines = ReadFile.OLines
        self.OHeaderLastLine = ReadFile.OHeaderLastLine
        self.ApproxPos = ReadFile.ApproxPos
        self.hatchfilter = HatchFilter(0.5)
        self.sat_filters = {}
        self.final_positions = []
        self.gdop_values = []
        self.pdop_values = []
        self.hdop_values = []
        self.vdop_values = []
        self.tdop_values = []

```

```

        self.observation_times = []

    def GenerateObs(self):
        return 0

    def MatchObservationAndCaculate(self):
        line = self.Lines[self.OHeaderLastLine]
        NReadLine = self.OHeaderLastLine
        line_count = len(self.Lines)
        while NReadLine < line_count and line:
            obs_time = [None] * 6
            obs_time[0] = int((line[1:3]).strip()) +
2000
            obs_time[1] = int((line[4:6]).strip())
            obs_time[2] = int((line[7:9]).strip())
            obs_time[3] = int((line[10:12]).strip())
            obs_time[4] = int((line[13:15]).strip())
            obs_time[5] = int((line[17:18]).strip())
            num_sat = int(line[30:32])
            if (num_sat % 12 == 0):
                n = int(num_sat / 12)
            else:
                n = int((num_sat / 12)) + 1
            sat_str = ''
            for j in range(n):
                if NReadLine + j < line_count:
                    sat_str = sat_str +
self.Lines[NReadLine + j][32:68].strip()
                obs_sat_PRN = []
                for k in range(num_sat):
                    obs_sat_PRN.append(sat_str[k * 3:k * 3 +
3])

            NReadLine = NReadLine + n
            ObsPseudorange = []
            for j in range(NReadLine, NReadLine + 2 *
num_sat, 2):
                if j < line_count:
                    line = self.Lines[j]

```

```

        if (line[34:46].strip() == ""):
            Pseudorange = 0
        else:
            Pseudorange = float(line[34:46])
            ObsPseudorange.append(Pseudorange)
    ObsCarrier = []
    for j in range(NReadLine, NReadLine + 2 *
num_sat, 2):
        if j < line_count:
            line = self.Lines[j]
            if (line[46:58].strip() == ""):
                Carrier = 0
            else:
                Carrier = float(line[50:63])
                ObsCarrier.append(Carrier)
    SmoothPseudorange = []
    for j in range(num_sat):
        prn = obs_sat_PRN[j]
        carrier = ObsCarrier[j]
        pseudorange = ObsPseudorange[j]
        if prn not in self.sat_filters:
            self.sat_filters[prn] =
HatchFilter(0.5)
            filter_instance = self.sat_filters[prn]
            smoothed =
filter_instance.filter(carrier, pseudorange)
            SmoothPseudorange.append(smoothed)
            SatLiteXYZ = self.MatchToSatlite(obs_time,
obs_sat_PRN)
            a =
self.SolutionLeastSquares(SmoothPseudorange,
SatLiteXYZ, ReadFile.ApproxPos, obs_time)
            NReadLine = NReadLine + 2 * num_sat
            if NReadLine < line_count:
                line = self.Lines[NReadLine]
            else:
                break

```

```

def MatchToSatlite(self, ObsTime, ObsSatPrn):
    SatLiteXYZ = []
    for index, SatPRN in enumerate(ObsSatPrn):
        TemXYZ = [None] * 5
        TimeDiff = []
        for index1, SatPRN1 in
enumerate(self.SateliteName):
            if (SatPRN == SatPRN1):
                TimeDiff.append(self.CaculateTimeDiff
erence(ObsTime, self.Time[index1]))
            else:
                TimeDiff.append(2592000)
        NotMatch = all(x == 2592000 for x in
TimeDiff)
        if (NotMatch == True):
            TemXYZ[0] = 0
            TemXYZ[1] = 0
            TemXYZ[2] = 0
            TemXYZ[3] = 0
            TemXYZ[4] = 0
        else:
            MinTime = min(TimeDiff)
            MinTimeindex = TimeDiff.index(MinTime)
            satelite =
Satelite(self.SateliteName[MinTimeindex],
self.Time[MinTimeindex],
                self.SateliteClockCorre
ct[MinTimeindex],
self.SateliteObservation[MinTimeindex])
            satelite.InitPositionOfSat(ObsTime)
            TemXYZ[0] = satelite.X
            TemXYZ[1] = satelite.Y
            TemXYZ[2] = satelite.Z
            TemXYZ[3] = 1
            TemXYZ[4] = satelite.Delta_T
            SatLiteXYZ.append(TemXYZ)
    return SatLiteXYZ

```

```

    def CaculateTimeDifference(self, Time1, Time2):
        sec1 = int(Time1[5]) if isinstance(Time1[5],
int) else Time1[5]
        sec2 = int(Time2[5]) if isinstance(Time2[5],
int) else Time2[5]
        time1 = datetime.datetime(Time1[0], Time1[1],
Time1[2], Time1[3], Time1[4], int(sec1),
                                microsecond=int((sec1 -
int(sec1)) * 1e6))
        time2 = datetime.datetime(Time2[0], Time2[1],
Time2[2], Time2[3], Time2[4], int(sec2),
                                microsecond=int((sec2 -
int(sec2)) * 1e6))
        diff = time2 - time1
        return diff.total_seconds()

    def SolutionLeastSquares(self, SmoothPseudorange,
SatLiteXYZ, ApproxPos, obs_time):
        t = 0
        B = []
        L = []
        SizeSatLiteXYZ = len(SatLiteXYZ)
        for i in range(SizeSatLiteXYZ):
            t = t + SatLiteXYZ[i][3]
        if (t < 4):
            return 0
        else:
            try:
                # 计算矩阵的条件数
                for k in range(SizeSatLiteXYZ):
                    if (SatLiteXYZ[k][3] == 0):
                        continue
                    if (k == SizeSatLiteXYZ - 1):
                        break
                    P0 = np.sqrt(
                        np.square(SatLiteXYZ[k][0] -
ApproxPos[0]) + np.square(

```

```

SatLiteXYZ[k][1] -
ApproxPos[1])) + np.square(
SatLiteXYZ[k][2] -
ApproxPos[2]))
    TemB0 = -1 * (SatLiteXYZ[k][0] -
ApproxPos[0]) / P0
    TemB1 = -1 * (SatLiteXYZ[k][1] -
ApproxPos[1]) / P0
    TemB2 = -1 * (SatLiteXYZ[k][2] -
ApproxPos[2]) / P0
    c = 3 * (10 ** -8)
    TemB3 = 1
    TemBRol = [TemB0, TemB1, TemB2,
TemB3]
    B.append(TemBRol)
    TemLRol = SmoothPseudorange[k] - P0 +
c * SatLiteXYZ[k][4]
    L.append(TemLRol)
    ArrayB = np.array(B)
    ArrayL = np.array(L)
    condition_number =
np.linalg.cond(np.transpose(ArrayB) @ ArrayB)
    if condition_number > 1e12: # 条件数过
大，使用伪逆
        x =
np.linalg.pinv(np.transpose(ArrayB) @ ArrayB) @
(np.transpose(ArrayB) @ ArrayL)
    else:
        x =
np.linalg.inv(np.transpose(ArrayB) @ ArrayB) @
(np.transpose(ArrayB) @ ArrayL)
    except np.linalg.LinAlgError:
        print("矩阵奇异，使用伪逆计算")
        x = np.linalg.pinv(np.transpose(ArrayB) @
ArrayB) @ (np.transpose(ArrayB) @ ArrayL)
    PosXYZ = np.array(ApproxPos)
    updated_pos = [
        PosXYZ[0] + x[0],

```

```

        PosXYZ[1] + x[1],
        PosXYZ[2] + x[2]
    ]
    self.final_positions.append(updated_pos)
    GDOP, PDOP, HDOP, VDOP, TDOP =
self.Accuracy(SatLiteXYZ, ApproxPos)
    self.gdop_values.append(GDOP)
    self.pdop_values.append(PDOP)
    self.hdop_values.append(HDOP)
    self.vdop_values.append(VDOP)
    self.tdop_values.append(TDOP)
    current_time = f"{obs_time[0]}-
{obs_time[1]}-{obs_time[2]}
{obs_time[3]}:{obs_time[4]}:{obs_time[5]}"
    self.observation_times.append(current_time)
    return 1

def Accuracy(self, SatLiteXYZ, ApproxPos):
    M = np.zeros((len(SatLiteXYZ), 4))
    for i in range(len(SatLiteXYZ)):
        if (SatLiteXYZ[i][3] == 0):
            continue
        M[i][0] = -1 * (SatLiteXYZ[i][0] -
ApproxPos[0])
        M[i][1] = -1 * (SatLiteXYZ[i][1] -
ApproxPos[1])
        M[i][2] = -1 * (SatLiteXYZ[i][2] -
ApproxPos[2])
        M[i][3] = 1
    try:
        Q = np.linalg.inv(np.transpose(M) @ M)
    except np.linalg.LinAlgError:
        print("计算精度指标时矩阵奇异，使用伪逆计算")
        Q = np.linalg.pinv(np.transpose(M) @ M)
    GDOP = np.sqrt(np.trace(Q))
    PDOP = np.sqrt(Q[0][0] + Q[1][1] + Q[2][2])
    HDOP = np.sqrt(Q[0][0] + Q[1][1])
    VDOP = np.sqrt(Q[2][2])

```

```
TDOP = np.sqrt(Q[3][3])
return GDOP, PDOP, HDOP, VDOP, TDOP
```

ReadFile.py

```
import numpy as np
from Satelite import Satelite

class ReadFile:
    ApproxPos = [None] * 3
    NLines = None
    OLines = None
    OHeaderLastLine = 0

    def __init__(self, File):
        self.oFilePath = File[0]
        self.nFilePath = File[1]
        self.OLines_ = self.ReadOFile()
        self.NLines_ = self.ReadNFile()
        self.ApproxPos = []
        self.NHeaderLastLine =
ReadFile.PreprocessNFile(self.NLines_)
        self.OHeaderLastLine =
ReadFile.PreprocessOFile(self, self.OLines_)
        if ReadFile.NLines is None:
            ReadFile.NLines = self.NLines_
        if ReadFile.OLines is None:
            ReadFile.OLines = self.OLines_
        if ReadFile.ApproxPos == [None] * 3:
            ReadFile.ApproxPos = self.ApproxPos
        if ReadFile.OHeaderLastLine == 0:
            ReadFile.OHeaderLastLine =
self.OHeaderLastLine
        self.Satelites = []
        self.Pos = []
        self.PosName = []
        self.Time = []
        self.RefTime = []
        self.SateliteClockCorrect = []
```

```

        self.SateliteObservation = []

    @classmethod
    def GetApproxPos(self):
        return self.ApproxPos

    @classmethod
    def GetOLines(self):
        return self.OLines

    @classmethod
    def GetOHeaderLastLine(self):
        return self.OHeaderLastLine

    @classmethod
    def GetNlines(self):
        return self.NLines

    @classmethod
    def GetNHeaderLastLine(self):
        return self.NHeaderLastLine

    def ReadNFile(self):
        with open(self.nFilePath, 'r') as file:
            lines = file.readlines()
            ReadFile.NLines = lines
            return lines

    def ReadOFile(self):
        with open(self.oFilePath, 'r') as file:
            lines = file.readlines()
            ReadFile.OLines = lines
            return lines

    @staticmethod
    def PreprocessNFile(lines):
        target_string = 'END OF HEADER'
        HeadLine = 0

```

```

        for i, line in enumerate(lines, start=1):
            if target_string in line:
                HeadLine = i
                break
        return HeadLine

    def PreprocessOFile(self, lines):
        ApproxPosComment = 'APPROX POSITION XYZ'
        for i, line in enumerate(lines, start=1):
            if (ApproxPosComment in line):
                approx_x = float(line[0:15].strip())
                approx_y = float(line[15:28].strip())
                approx_z = float(line[29:42].strip())
                self.ApproxPos.append(approx_x)
                self.ApproxPos.append(approx_y)
                self.ApproxPos.append(approx_z)
        ObsTargetString = 'END OF HEADER'
        ObsHeaderLine = 0
        for i, line in enumerate(lines, start=1):
            if (ObsTargetString in line):
                ObsHeaderLine = i
                break
        return ObsHeaderLine

    @staticmethod
    def CaculateSatRefTime(Time):
        return 0

    def CaculateSatelites(self):
        for i in range(self.NHeaderLastLine,
len(self.NLines) - 9, 8):
            line = self.NLines[i]
            num = line[0:2].strip()
            time = [None] * 6
            time[0] = int((line[3:5]).strip()) + 2000
            time[1] = int((line[6:8]).strip())
            time[2] = int((line[9:11]).strip())
            time[3] = int((line[12:14]).strip())

```

```

        time[4] = int((line[15:17]).strip())
        time[5] = float((line[18:22]).strip())
        time_change = []
        a = float(line[22:37].strip()) * pow(10,
int(line[38:41].strip()))
        time_change.append(a)
        b = float(line[41:56].strip()) * pow(10,
int(line[57:60].strip()))
        time_change.append(b)
        c = float(line[60:75].strip()) * pow(10,
int(line[76:79].strip()))
        time_change.append(c)
        self.SateliteClockCorrect.append(time_change
)

        rows = 6
        cols = 4
        matrix = np.zeros((rows, cols))
        for j in range(0, rows):
            for k in range(0, cols):
                matrix[j][k] = float(self.NLines[i +
1 + j][3 + 19 * k:18 + 19 * k]) * pow(10, int(
                self.NLines[i + 1 + j][19 + 19 *
k:22 + 19 * k]))
            self.SateliteObservation.append(matrix)
            SatelliteName = "G" + str(num)
            satelite = Satelite(SateliteName, time,
time_change, matrix)
            satelite.InitPositionOfSat(time)
            self.Pos.append([satelite.X, satelite.Y,
satelite.Z])
            self.Time.append(time)
            self.PosName.append(SateliteName)
            self.Satelites.append(satelite)

```

Result.py

```

import pandas as pd

def save_results_to_excel(position, file_name):

```

```

        if position.final_positions:
            data1 = {
                'Observation Time':
position.observation_times,
                'X': [pos[0] for pos in
position.final_positions],
                'Y': [pos[1] for pos in
position.final_positions],
                'Z': [pos[2] for pos in
position.final_positions],
            }
            data2 = {
                'Observation Time':
position.observation_times,
                'GDOP': position.gdop_values,
                'PDOP': position.pdop_values,
                'HDOP': position.hdop_values,
                'VDOP': position.vdop_values,
                'TDOP': position.tdop_values
            }
            df = pd.DataFrame(data1)
            csv_file_name1 = "./output/xyz.csv"
            df.to_csv(csv_file_name1, index=False)
            df = pd.DataFrame(data2)
            csv_file_name2 = "./output/dop.csv"
            df.to_csv(csv_file_name2, index=False)
        else:
            print("没有有效的定位结果可以保存到 Excel 文件中。")

```

Satelite.py

```

import datetime
import numpy as np

class Satelite:
    def __init__(self, SateliteName, Time,
SateliteClockCorrect, SateliteObservation):
        self.SateliteName = SateliteName
        self.GpsSeconds = Satelite.ctime2gps(Time)

```

```

        self.Time = Time
        self.SateliteRefTime = Time
        self.SateliteClockCorrect = SateliteClockCorrect
        self.SateliteObservation = SateliteObservation
        self.Delta_T = 0

    def InitPositionOfSat(self, ObsTime):
        [sat_x, sat_y, sat_z, delta_t_value] =
Satelite.caculate_pos_of_sat(self.SateliteObservation,

                               self.SateliteClockCorrect, ObsTime,

                               self.Time)
        self.X = sat_x
        self.Y = sat_y
        self.Z = sat_z
        self.Delta_T = delta_t_value

    @classmethod
    def GetPositionOfSat(self):
        return self.X, self.Y, self.Z

    @staticmethod
    def ctime2gps(time):
        gps_epoch = datetime.datetime(1980, 1, 6, 0, 0,
0)
        given_time = datetime.datetime(time[0], time[1],
time[2], time[3], time[4], int(time[5]))
        time_diff = given_time - gps_epoch
        total_seconds = time_diff.total_seconds()
        gps_week, gps_seconds = divmod(total_seconds,
604800)
        return gps_week, gps_seconds

    @staticmethod
    def caculate_pos_of_sat(matrix, a3, obstime,
reftime):
        GM = 3.986005 * pow(10, 5)

```

```

n0 = np.sqrt(GM) / pow(matrix[1, 3], 3)
n = n0 + matrix[0, 2]
gps_obsweek, gps_obssec =
Satelite.ctime2gps(obstime)
gps_refweek, gps_refseconds =
Satelite.ctime2gps(reftime)
delta_t = a3[0] + a3[1] * (gps_obssec -
gps_refseconds) + a3[2] * pow(gps_obssec -
gps_refseconds, 2)
t = gps_refseconds - delta_t
tk = t - matrix[2, 0]
Mk = matrix[0, 3] + n * tk
Mk = Mk % (np.pi * 2)
Ek = Mk
Ek_old = Ek
for i in range(10):
    Ek = Mk + matrix[1, 1] * np.sin(Ek)
    if (abs(Ek - Ek_old) < 1e-10):
        break
    else:
        Ek_old = Ek
    cos_Vk = (np.cos(Ek) - matrix[1, 1]) / (1 -
matrix[1, 1] * np.cos(Ek))
    sin_Vk = (np.sqrt(1 - matrix[1, 1] * matrix[1,
1]) * np.sin(Ek)) / (1 - matrix[1, 1] * np.cos(Ek))
    Vk = np.arctan(np.sqrt(1 - matrix[1, 1] *
matrix[1, 1]) * np.sin(Ek) / (np.cos(Ek) - matrix[1,
1]))
    phi_k = Vk + matrix[3, 2]
    phi_k = phi_k % (np.pi * 2)
    delta_u = matrix[1, 0] * np.cos(2 * phi_k) +
matrix[1, 2] * np.sin(2 * phi_k)
    delta_r = matrix[3, 1] * np.cos(2 * phi_k) +
matrix[0, 1] * np.sin(2 * phi_k)
    delta_i = matrix[2, 1] * np.cos(2 * phi_k) +
matrix[2, 3] * np.sin(2 * phi_k)
    uk = phi_k + delta_u

```

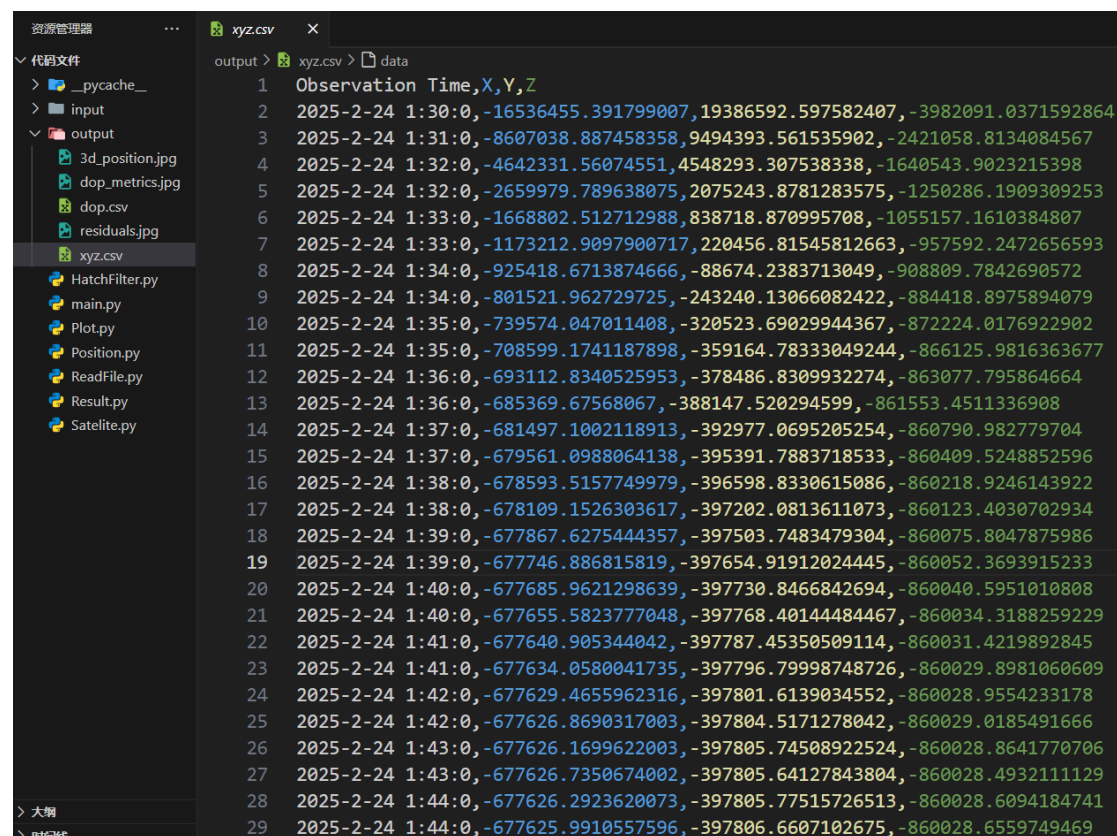
```

        rk = pow(matrix[1, 3], 2) * (1 - matrix[1, 1] *
np.cos(Ek)) + delta_r
        ik = matrix[3, 0] + delta_i + matrix[4, 0] * tk
        xk = rk * np.cos(uk)
        yk = rk * np.sin(uk)
        we = 7.29211567 * pow(10, -5)
        omegkek = matrix[2, 2] + (matrix[3, 3] - we) * tk
- we * matrix[2, 0]
        Xk = xk * np.cos(omegek) - yk * np.cos(ik) *
np.sin(omegek)
        Yk = xk * np.sin(omegek) + yk * np.cos(ik) *
np.cos(omegek)
        Zk = yk * np.sin(ik)
        return Xk, Yk, Zk, delta_t

```

三、 结果

经过计算，我们计算得到了 2025 年 2 月 24 日各个星历时刻巴黎测站的坐标值 xyz.csv 文件，如下图所示：



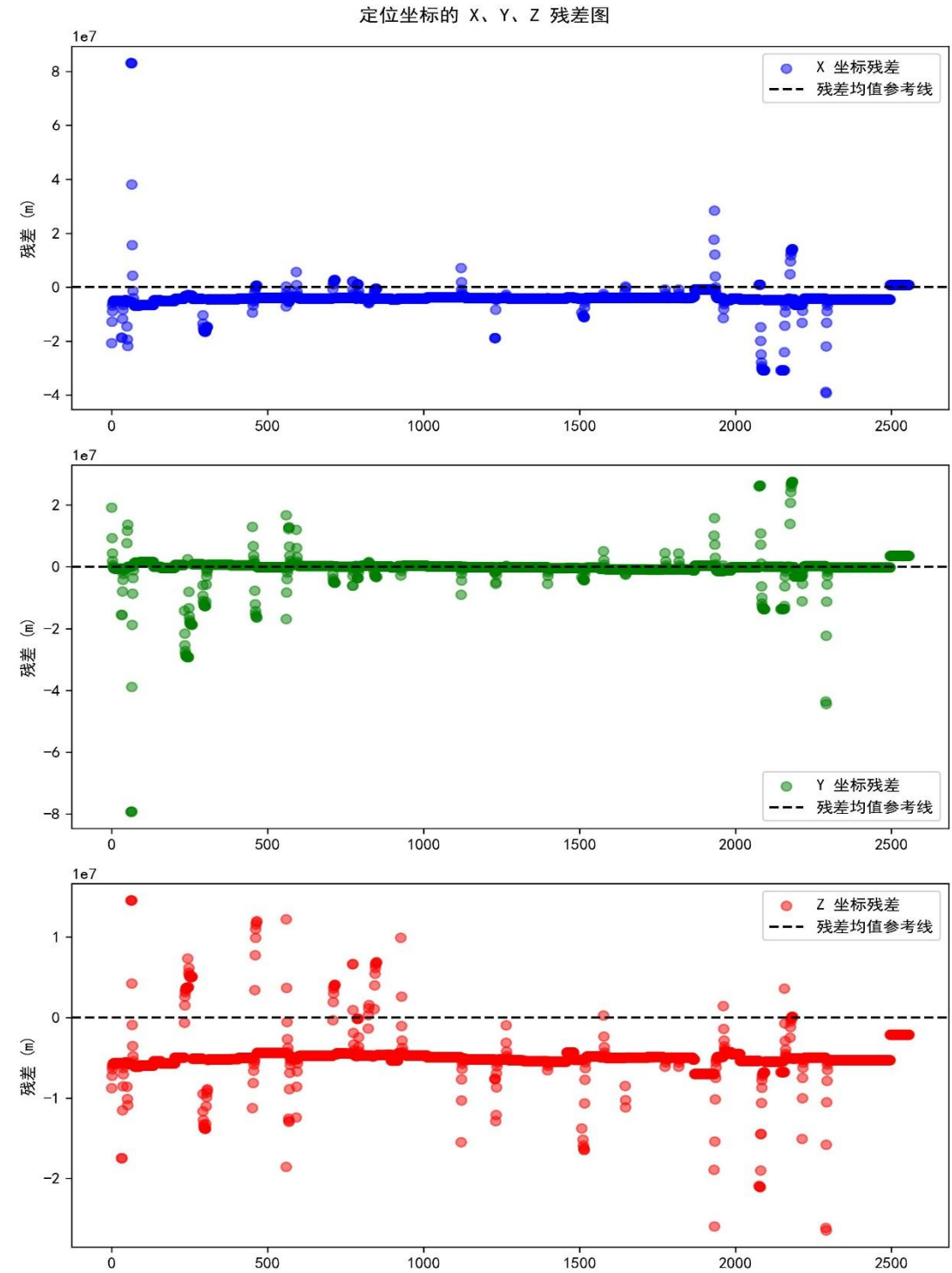
Observation Time	X	Y	Z	delta_t
2025-2-24 1:30:0	-16536455.391799007	19386592.597582407	-3982091.0371592864	
2025-2-24 1:31:0	-8607038.887458358	9494393.561535902	-2421058.8134084567	
2025-2-24 1:32:0	-4642331.56074551	4548293.307538338	-1640543.9023215398	
2025-2-24 1:32:0	-2659979.789638075	2075243.8781283575	-1250286.1909309253	
2025-2-24 1:33:0	-1668802.512712988	838718.870995708	-1055157.1610384807	
2025-2-24 1:33:0	-1173212.9097900717	220456.81545812663	-957592.2472656593	
2025-2-24 1:34:0	-925418.6713874666	-88674.2383713049	-908809.7842690572	
2025-2-24 1:34:0	-801521.962729725	-243240.13066082422	-884418.8975894079	
2025-2-24 1:35:0	-739574.047011408	-320523.69029944367	-872224.0176922902	
2025-2-24 1:35:0	-708599.1741187898	-359164.78333049244	-866125.9816363677	
2025-2-24 1:36:0	-693112.8340525953	-378486.8309932274	-863077.795864664	
2025-2-24 1:36:0	-685369.67568067	-388147.520294599	-861553.4511336908	
2025-2-24 1:37:0	-681497.1002118913	-392977.0695205254	-860790.982779704	
2025-2-24 1:37:0	-679561.0988064138	-395391.7883718533	-860409.5248852596	
2025-2-24 1:38:0	-678593.5157749979	-396598.8330615086	-860218.9246143922	
2025-2-24 1:38:0	-678109.1526303617	-397202.0813611073	-860123.4030702934	
2025-2-24 1:39:0	-677867.6275444357	-397503.7483479304	-860075.8047875986	
2025-2-24 1:39:0	-677746.886815819	-397654.91912024445	-860052.3693915233	
2025-2-24 1:40:0	-677685.9621298639	-397730.8466842694	-860040.5951010808	
2025-2-24 1:40:0	-677655.5823777048	-397768.40144484467	-860034.3188259229	
2025-2-24 1:41:0	-677640.905344042	-397787.45350509114	-860031.4219892845	
2025-2-24 1:41:0	-677634.0580041735	-397796.79998748726	-860029.8981060609	
2025-2-24 1:42:0	-677629.4655962316	-397801.6139034552	-860028.9554233178	
2025-2-24 1:42:0	-677626.8690317003	-397804.5171278042	-860029.0185491666	
2025-2-24 1:43:0	-677626.1699622003	-397805.74508922524	-860028.8641770706	
2025-2-24 1:43:0	-677626.7350674002	-397805.64127843804	-860028.4932111129	
2025-2-24 1:44:0	-677626.2923620073	-397805.77515726513	-860028.6094184741	
2025-2-24 1:44:0	-677625.9910557596	-397806.6607102675	-860028.6559749469	

同时，我们可以计算得到对应的各个星历计算值的 GDOP, PDOP, HDOP, VDOP, TDOP 值，输出为 dop.csv 文件，如图：

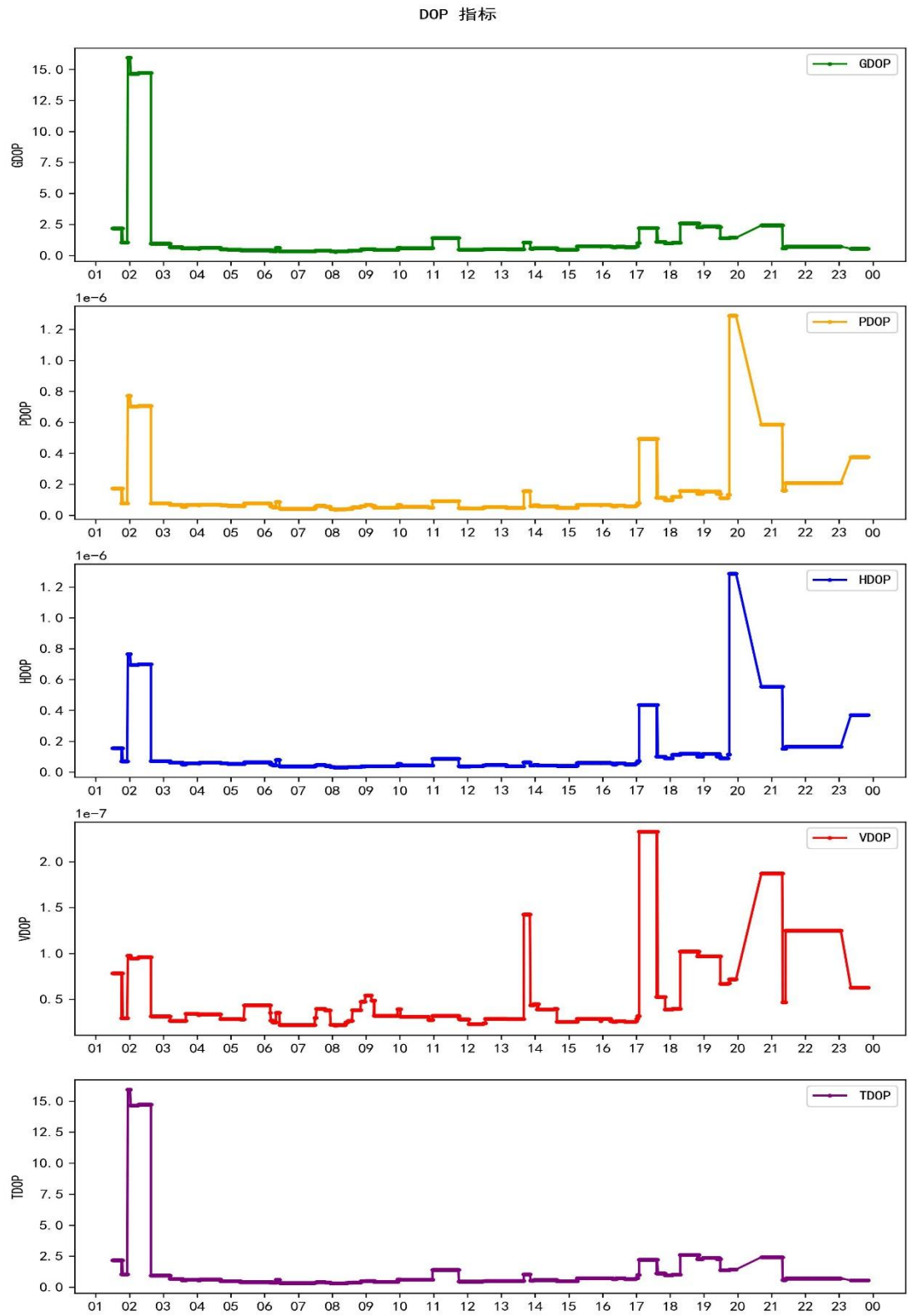
```
Observation Time,GDOP,PDOP,HDOP,VDOP,TDOP
2025-2-24 1:30:0,2.1823794064401816,1.7381377936844937e-07,1.5507697767341023e-07,7.850070632818931e-08,2.1823794064401745
2025-2-24 1:31:0,2.1823794064402184,1.738137793684554e-07,1.5507697767341486e-07,7.850070632819353e-08,2.1823794064402113
2025-2-24 1:32:0,2.1823794064402344,1.7381377936846e-07,1.5507697767341817e-07,7.850070632819714e-08,2.1823794064402278
2025-2-24 1:32:0,2.1823794064402344,1.7381377936846e-07,1.5507697767341817e-07,7.850070632819714e-08,2.1823794064402278
2025-2-24 1:33:0,2.1823794064402646,1.7381377936846544e-07,1.5507697767342227e-07,7.850070632820111e-08,2.1823794064402575
2025-2-24 1:33:0,2.1823794064402646,1.7381377936846544e-07,1.5507697767342227e-07,7.850070632820111e-08,2.1823794064402575
2025-2-24 1:34:0,2.1823794064403015,1.738137793684714e-07,1.550769776734268e-07,7.850070632820531e-08,2.182379406440295
2025-2-24 1:34:0,2.1823794064403015,1.738137793684714e-07,1.550769776734268e-07,7.850070632820531e-08,2.182379406440295
2025-2-24 1:35:0,2.182379406440334,1.7381377936847708e-07,1.5507697767343114e-07,7.850070632820939e-08,2.182379406440327
2025-2-24 1:35:0,2.182379406440334,1.7381377936847708e-07,1.5507697767343114e-07,7.850070632820939e-08,2.182379406440327
2025-2-24 1:36:0,2.182379406440359,1.7381377936848224e-07,1.5507697767343495e-07,7.850070632821328e-08,2.182379406440352
2025-2-24 1:36:0,2.182379406440359,1.7381377936848224e-07,1.5507697767343495e-07,7.850070632821328e-08,2.182379406440352
2025-2-24 1:37:0,2.1823794064403823,1.738137793684875e-07,1.5507697767343884e-07,7.850070632821726e-08,2.182379406440375
2025-2-24 1:37:0,2.1823794064403823,1.738137793684875e-07,1.5507697767343884e-07,7.850070632821726e-08,2.182379406440375
2025-2-24 1:38:0,2.182379406440424,1.738137793684937e-07,1.5507697767344363e-07,7.85007063282215e-08,2.182379406440417
2025-2-24 1:38:0,2.182379406440424,1.738137793684937e-07,1.5507697767344363e-07,7.85007063282215e-08,2.182379406440417
2025-2-24 1:39:0,2.182379406440454,1.7381377936849937e-07,1.5507697767344787e-07,7.850070632822567e-08,2.182379406440467
2025-2-24 1:39:0,2.182379406440454,1.7381377936849937e-07,1.5507697767344787e-07,7.850070632822567e-08,2.182379406440467
2025-2-24 1:40:0,2.182379406440477,1.7381377936850464e-07,1.5507697767345176e-07,7.850070632822962e-08,2.1823794064404702
2025-2-24 1:40:0,2.182379406440477,1.7381377936850464e-07,1.5507697767345176e-07,7.850070632822962e-08,2.1823794064404702
2025-2-24 1:41:0,2.1823794064405115,1.7381377936851038e-07,1.5507697767345613e-07,7.850070632823373e-08,2.1823794064405044
2025-2-24 1:41:0,2.1823794064405115,1.7381377936851038e-07,1.5507697767345613e-07,7.850070632823373e-08,2.1823794064405044
2025-2-24 1:42:0,2.1823794064405346,1.7381377936851544e-07,1.5507697767345986e-07,7.850070632823755e-08,2.1823794064405275
2025-2-24 1:42:0,2.1823794064405346,1.7381377936851544e-07,1.5507697767345986e-07,7.850070632823755e-08,2.1823794064405275
2025-2-24 1:43:0,2.1823794064405737,1.738137793685215e-07,1.5507697767346452e-07,7.850070632824176e-08,2.182379406440567
2025-2-24 1:43:0,2.1823794064405737,1.738137793685215e-07,1.5507697767346452e-07,7.850070632824176e-08,2.182379406440567
2025-2-24 1:44:0,2.182379406440604,1.738137793685272e-07,1.550769776734688e-07,7.850070632824589e-08,2.182379406440597
2025-2-24 1:44:0,2.182379406440604,1.738137793685272e-07,1.550769776734688e-07,7.850070632824589e-08,2.182379406440597
2025-2-24 1:45:0,2.182379406440629,1.738137793685323e-07,1.550769776734726e-07,7.850070632824971e-08,2.182379406440622
2025-2-24 1:45:0,2.182379406440629,1.738137793685323e-07,1.550769776734726e-07,7.850070632824971e-08,2.182379406440622
2025-2-24 1:46:0,2.182379406440664,1.7381377936853836e-07,1.550769776734772e-07,7.850070632825407e-08,2.1823794064406568
2025-2-24 1:46:0,2.182379406440664,1.7381377936853836e-07,1.550769776734772e-07,7.850070632825407e-08,2.1823794064406568
2025-2-24 1:47:0,2.182379406440689,1.7381377936854357e-07,1.5507697767348106e-07,7.850070632825797e-08,2.182379406440682
2025-2-24 1:47:0,2.182379406440689,1.7381377936854357e-07,1.5507697767348106e-07,7.850070632825797e-08,2.182379406440682
2025-2-24 1:48:0,2.182379406440689,1.7381377936854357e-07,1.5507697767348106e-07,7.850070632825797e-08,2.182379406440682
2025-2-24 1:48:0,2.182379406440689,1.7381377936854357e-07,1.5507697767348106e-07,7.850070632825797e-08,2.182379406440682
```

```
1 Observation Time,GDOP,PDOP,HDOP,VDOP,TDOP
2520 2025-2-24 23:35:0,0.551060655430937,3.7497038245113254e-07,3.696863980430838e-07,6.272762403848146e-08,0.5510606554308094
2521 2025-2-24 23:35:0,0.551060655430937,3.7497038245113254e-07,3.696863980430838e-07,6.272762403848146e-08,0.5510606554308094
2522 2025-2-24 23:36:0,0.5510606554309415,3.749703824513746e-07,3.6968639804327913e-07,6.272762403850715e-08,0.551060655430814
2523 2025-2-24 23:36:0,0.5510606554309415,3.749703824513746e-07,3.6968639804327913e-07,6.272762403850715e-08,0.551060655430814
2524 2025-2-24 23:37:0,0.5510606554309461,3.749703824516087e-07,3.696863980435124e-07,6.272762403853162e-08,0.5510606554308185
2525 2025-2-24 23:37:0,0.5510606554309461,3.749703824516087e-07,3.696863980435124e-07,6.272762403853162e-08,0.5510606554308185
2526 2025-2-24 23:38:0,0.5510606554309507,3.74970382451852e-07,3.696863980437549e-07,6.27276240385573e-08,0.5510606554308232
2527 2025-2-24 23:38:0,0.5510606554309507,3.74970382451852e-07,3.696863980437549e-07,6.27276240385573e-08,0.5510606554308232
2528 2025-2-24 23:39:0,0.5510606554309553,3.7497038245209323e-07,3.696863980439953e-07,6.272762403858269e-08,0.5510606554308277
2529 2025-2-24 23:39:0,0.5510606554309553,3.7497038245209323e-07,3.696863980439953e-07,6.272762403858269e-08,0.5510606554308277
2530 2025-2-24 23:40:0,0.55106065543096,3.7497038245233247e-07,3.696863980442336e-07,6.272762403860819e-08,0.5510606554308324
2531 2025-2-24 23:40:0,0.55106065543096,3.7497038245233247e-07,3.696863980442336e-07,6.272762403860819e-08,0.5510606554308324
2532 2025-2-24 23:41:0,0.5510606554309646,3.7497038245257054e-07,3.696863980444708e-07,6.272762403863323e-08,0.551060655430837
2533 2025-2-24 23:41:0,0.5510606554309646,3.7497038245257054e-07,3.696863980444708e-07,6.272762403863323e-08,0.551060655430837
2534 2025-2-24 23:42:0,0.551060655430969,3.7497038245281273e-07,3.696863980447121e-07,6.272762403865884e-08,0.5510606554308415
2535 2025-2-24 23:42:0,0.551060655430969,3.7497038245281273e-07,3.696863980447121e-07,6.272762403865884e-08,0.5510606554308415
2536 2025-2-24 23:43:0,0.5510606554309737,3.7497038245304863e-07,3.696863980449472e-07,6.272762403868348e-08,0.5510606554308461
2537 2025-2-24 23:43:0,0.5510606554309737,3.7497038245304863e-07,3.696863980449472e-07,6.272762403868348e-08,0.5510606554308461
2538 2025-2-24 23:44:0,0.5510606554309783,3.749703824532923e-07,3.6968639804519e-07,6.272762403870936e-08,0.5510606554308506
2539 2025-2-24 23:44:0,0.5510606554309783,3.749703824532923e-07,3.6968639804519e-07,6.272762403870936e-08,0.5510606554308506
2540 2025-2-24 23:45:0,0.5510606554309829,3.749703824535288e-07,3.696863980454256e-07,6.272762403873425e-08,0.5510606554308554
2541 2025-2-24 23:45:0,0.5510606554309829,3.749703824535288e-07,3.696863980454256e-07,6.272762403873425e-08,0.5510606554308554
2542 2025-2-24 23:46:0,0.5510606554309875,3.7497038245376517e-07,3.6968639804566114e-07,6.27276240387593e-08,0.5510606554308599
2543 2025-2-24 23:46:0,0.5510606554309875,3.7497038245376517e-07,3.6968639804566114e-07,6.27276240387593e-08,0.5510606554308599
2544 2025-2-24 23:47:0,0.5510606554309921,3.7497038245400636e-07,3.696863980459015e-07,6.272762403878469e-08,0.5510606554308646
2545 2025-2-24 23:47:0,0.5510606554309921,3.7497038245400636e-07,3.696863980459015e-07,6.272762403878469e-08,0.5510606554308646
2546 2025-2-24 23:48:0,0.5510606554309967,3.7497038245424554e-07,3.6968639804613977e-07,6.27276240388098e-08,0.551060655430869
2547 2025-2-24 23:48:0,0.5510606554309967,3.7497038245424554e-07,3.6968639804613977e-07,6.27276240388098e-08,0.551060655430869
2548 2025-2-24 23:49:0,0.5510606554310014,3.749703824544841e-07,3.6968639804637747e-07,6.272762403883501e-08,0.5510606554308738
2549 2025-2-24 23:49:0,0.5510606554310014,3.749703824544841e-07,3.6968639804637747e-07,6.272762403883501e-08,0.5510606554308738
2550 2025-2-24 23:50:0,0.5510606554310059,3.749703824547218e-07,3.696863980466143e-07,6.272762403886013e-08,0.5510606554308782
2551 2025-2-24 23:50:0,0.5510606554310059,3.749703824547218e-07,3.696863980466143e-07,6.272762403886013e-08,0.5510606554308782
2552 2025-2-24 23:51:0,0.5510606554310106,3.7497038245496626e-07,3.6968639804685795e-07,6.272762403888578e-08,0.551060655430883
```

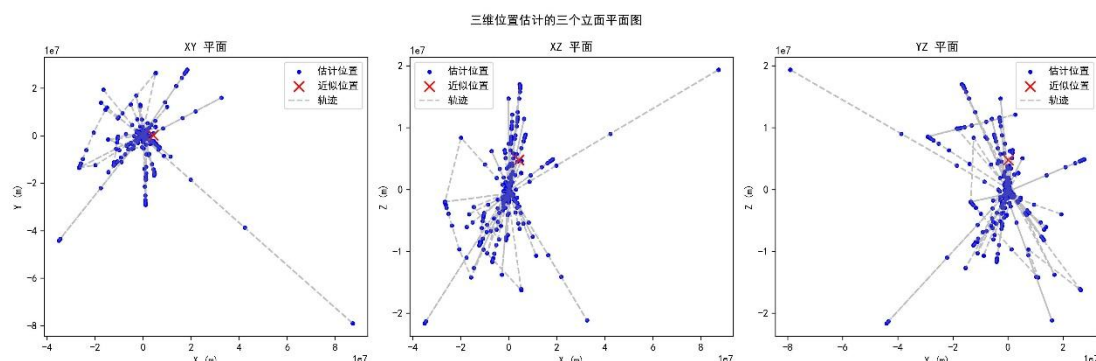
接着，我对 2025 年 2 月 24 日坐标的计算值进行误差分析，绘制了误差分布图，可以看到午夜时刻的误差较大，定位精度较低，其余时刻趋于平稳，定位精度较高。



继而，我绘制了 GDOP, PDOP, HDOP, VDOP, TDOP 的折线图，从图中也可以印证上述结论。



最后，为了形象展示该算法的定位效果，绘制了各个星历时刻三个立面的散点分布，改图可以直观形象的展示该算法的定位精度。



整体分析来看，该算法整体定位精度良好，定位精度整体较高，但在傍晚和凌晨的观测中，误差比较大，定位精度较低。引起偏差的原因可能是由于乌云遮挡或者大气条件的突变，导致卫星跟踪断开，产生周跳现象，导致定位精度在某些星历时刻突变。

四、 小结

在本次《卫星导航定位基础》课后作业实验中，围绕 GNSS 载波相位平滑伪距单点定位展开计算，旨在实现批量测站位置计算并评定精度，对比分析载波相位平滑伪距前后的定位精度变化。

从算法原理层面来看，伪距测量虽为 GNSS 定位基础，但易受多种误差影响，如卫星轨道误差、电离层延迟、对流层延迟以及多路径效应等，导致单纯依靠伪距测量的定位精度仅能达到米级。而载波相位测量精度潜力高，理论上可达毫米级，不过整周模糊度解算困难。本实验采用的载波相位平滑伪距算法，核心在于利用载波相位测量的高精度特性优化伪距测量结果。Hatch 滤波作为关键技

术，通过对连续的载波相位和伪距观测值进行加权平均，有效提升了伪距测量精度。其优势在于计算简单、易于实现，在稳定观测环境下能快速收敛，显著提高定位精度。但在信号遮挡严重或多路径效应强烈的环境中，算法存在局限性，会导致定位精度下降。

在实验过程中，选取了 2025 年 2 月 24 日巴黎测站的观测数据，利用 Python 语言在 3.13.2 运行环境下，借助 pandas、numpy 等多个库开展计算。通过 ReadFile.py 读取数据文件，进行数据预处理；HatchFilter.py 实现滤波算法；Position.py 计算接收站位置；Result.py 将结果输出保存；Plot.py 绘制各类图表用于分析。整个代码结构清晰，各模块分工明确，协同完成了从数据处理到结果呈现的全过程。

实验结果显示，成功计算出 2025 年 2 月 24 日巴黎测站各星历时刻的坐标值并保存于 xyz.csv 文件，同时得到对应的精度因子（GDOP、PDOP 等）值，输出为 dop.csv 文件。误差分析表明，午夜时刻误差较大、定位精度较低，其余时刻趋于平稳且定位精度较高。从绘制的误差分布图、精度因子折线图以及三个立面的散点分布图来看，该算法整体定位精度良好，但在傍晚和凌晨部分星历时刻存在精度突变的情况。推测可能是乌云遮挡、大气条件突变导致卫星跟踪断开，产生周跳现象，进而影响定位精度。

通过本次实验，深入理解了 GNSS 载波相位平滑伪距单点定位算法的原理、实现过程及其性能特点，提升了自我编程能力。